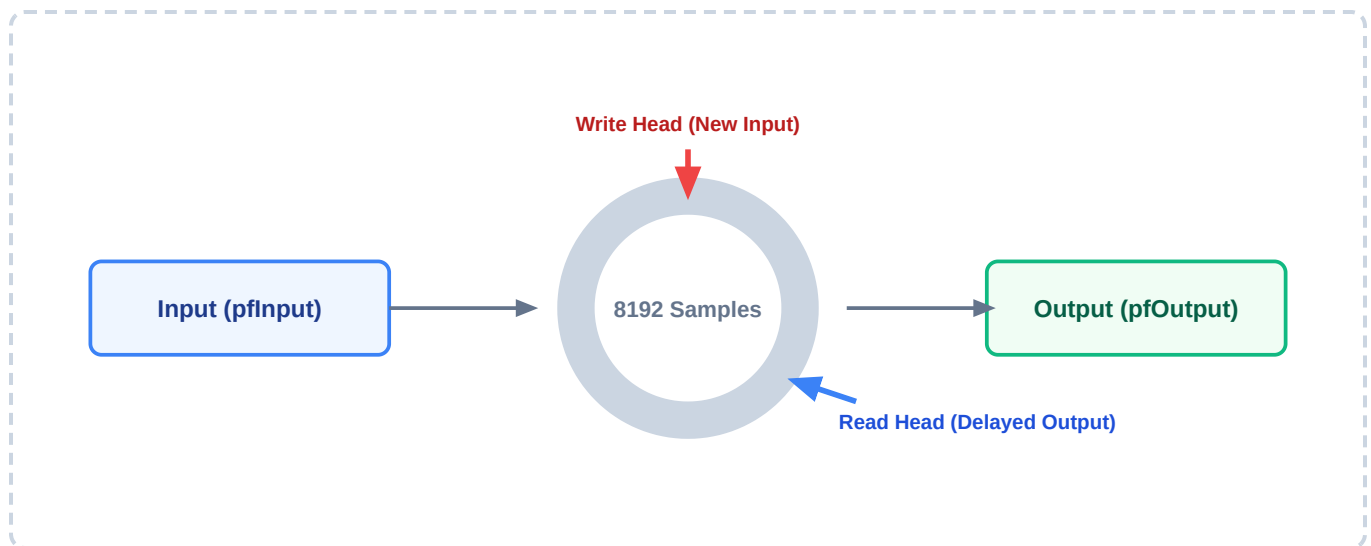


Audio Module Documentation: Delay Block

A simple breakdown of how the circular ring buffer creates a pure time-shift on our live microphone stream.

1. High-Level Diagram

This diagram tracks how our live data moves through a virtual circle in memory. Instead of moving audio data around (which is too slow for our ESP32), we leave the data alone and move our pointer index positions instead. The Read Pointer follows behind the Write Pointer at a fixed gap matching our desired delay time.



2. Module Structures & Memory Size

Our delay block allocates a fixed-size array space inside the memory structure to protect our system against memory leaks or audio crackles.

Configuration Structure (`delay_config_t`)

This structure simply carries our target delay window time value during our active initialization phase.

- `float delay_seconds` : The total amount of time we want to shift our signal to the right (e.g., 0.25f for a quarter-second shift).

Workspace Handle Structure (`delay_hdl_t`)

This holds our live state values and tracking variables inside our global pipeline memory area.

- `float delay_line[8192]` : Our dedicated ring buffer array. It holds up to 8192 historic audio samples.
- `uint32_t delay_samples` : The calculated layout gap distance between our read and write fingers.
- `uint32_t write_idx` : The active index where our new incoming microphone sample is placed.

Memory Usage Table

Internal Parameter Target	Data Type Class	Byte Weight Footprint	Dynamic Allocation (Malloc)
<code>phdl->delay_line</code>	8192 Float Array Elements	32,768 Bytes (32 KB)	0 Bytes (Statically Managed)
<code>phdl->delay_samples</code>	Unsigned 32-bit Integer	4 Bytes	0 Bytes (Statically Managed)

Internal Parameter Target	Data Type Class	Byte Weight Footprint	Dynamic Allocation (Malloc)
phdl->write_idx	Unsigned 32-bit Integer	4 Bytes	0 Bytes (Statically Managed)

3. Simple Mathematical Logic

The delay operates as a standard time-shift process. Every sample passing through is delayed by exactly D samples, which corresponds directly to our configured time window.

The mathematical rule implemented inside our processing loop is:

$$y[n] = x[n - D]$$

Where:

- **y[n]**: The outbound shifted audio signal value assigned to our output index.
- **x[n]**: The active real-time sample coming from our mic input stream.
- **D (delay_samples)**: The fixed tracking gap calculated based on our sample rate:
`D = (uint32_t)(delay_seconds * SAMPLE_RATE + 0.5f)`

4. Lifecycle Functions & Loop Execution

The system splits code execution pathways into standard lifecycle stages. As an architectural policy rule, API declarations do not automatically guarantee that sharing identical source and target arrays (in-place operation) is safe by default. Compatibility rests completely upon our internal processing loop sequence. This contract behavior must be explicitly stated inside the public documentation comments of our header prototype.

A. Query Memory Size (`delay_open`)

```
STATUS delay_open(uint32_t *pui32Size) {
    if (pui32Size == NULL) return STATUS_NOT_OK;
    *pui32Size = sizeof(delay_hdl_t);
    return STATUS_OK;
}
```

What it does: Tells our system exactly how much RAM space is needed to safely set up our handle variables before building our pipeline maps.

B. Initialize Module Settings (`delay_init`)

```
STATUS delay_init(delay_hdl_t *phdl, const delay_config_t *psConfig) {
    if (phdl == NULL || psConfig == NULL) return STATUS_NOT_OK;

    float delay_sec = psConfig->delay_seconds;
    if (delay_sec > MAX_DELAY_SECONDS) delay_sec = MAX_DELAY_SECONDS;
    if (delay_sec < 0.0f) delay_sec = 0.0f;

    phdl->delay_samples = (uint32_t)(delay_sec * SAMPLE_RATE + 0.5f);

    if (phdl->delay_samples >= MAX_DELAY_SAMPLES) phdl->delay_samples = MAX_DELAY_SAMPLES -
1;
    if (phdl->delay_samples == 0) phdl->delay_samples = 1;

    phdl->write_idx = 0;
    memset(phdl->delay_line, 0, sizeof(phdl->delay_line));

    return STATUS_OK;
}
```

What it does: Translates our chosen time value into an explicit number of samples, applies guardrails to keep variables within safe ranges, and clears out our 8192-sample buffer with zeros to avoid loud startup noise spikes.

C. Real-time Stream Engine Execution (`delay_process`)

```
/**
 * @brief Performs a pure signal time-shift using a circular buffer.
 * @note [NON-IN-PLACE COMPATIBLE] Since this loop updates the active
 * tracking position index of our shared ring buffer directly mid-loop,
 * passing the exact same pointer for pfInput and pfOutput will lead to
 * data clobbering. Separate buffers must be used.
 */
STATUS delay_process(delay_hdl_t *phdl, const float *pfInput, float *pfOutput, uint32_t
ui32NumSamples) {
    if (phdl == NULL || pfOutput == NULL) return STATUS_NOT_OK;

    for (uint32_t i = 0; i < ui32NumSamples; i++) {
        uint32_t read_idx = (phdl->write_idx + MAX_DELAY_SAMPLES - phdl->delay_samples) %
MAX_DELAY_SAMPLES;
        float delayed_sample = phdl->delay_line[read_idx];

        float input_sample = (pfInput != NULL) ? pfInput[i] : 0.0f;

        phdl->delay_line[phdl->write_idx] = input_sample;
        pfOutput[i] = delayed_sample;

        phdl->write_idx++;
        if (phdl->write_idx >= phdl->delay_samples) {
            phdl->write_idx = 0;
        }
    }
    return STATUS_OK;
}
```

What it does: Processes our streaming sample block. Because our loop updates our shared internal buffer values at the exact same index where we assign values to our destination registers, this function is **strictly non-in-place**. To protect your system from data clobbering, your input and output arguments must point to separate, isolated arrays.

D. Close and Clean Up (`delay_close`)

```
STATUS delay_close(delay_hdl_t *phdl) {  
    if (phdl == NULL) return STATUS_NOT_OK;  
    return STATUS_OK;  
}
```

What it does: Gracefully signs off the module handle. Since our structures are statically allocated across the scope, no runtime heap releasing loops are executed here.